College Football Bowl Game Predictor - Line-by-line code walkthrough

Files in the order they run: **main.py** starts, **data.py** loads data, **evaluate.py** runs everything, **feature_selection.py** picks features, **models.py** holds the models and the testing. `download_data.py` is only used once to build the CSV and is summarised at the end.

Words used below: **feature** = one statistic of a team; **margin** = score1 minus score2 (team 1's points minus team 2's); **X** = the input numbers; **y** = the answer we want to predict; **fit** = train the model; **predict** = use it on new data.

1. main.py (entry point)

Code	What it does
`"""Entry point..."""`	Docstring: a description of the file. It is not executed.
`import os` `import pandas as pd`	os is for folder paths; pandas (pd) is used to save tables as CSV.
`from data import load_data, FEATURES, SEASONS, RESULTS`	Brings in the data loader, the list of features, the list of seasons and the results folder path from data.py.
`from evaluate import run_all, make_figures`	Brings in the function that runs all models and the one that draws the charts.
`def main():`	Defines the main function that does the whole job.
`df, synthetic = load_data()`	Loads the dataset into a table df. synthetic is True if fake data was used, False for the real CSV.
`print(f"{len(df)} bowl games, {len(FEATURES)} features per team, seasons {df.season.min()}-{df.season.max()}")`	Prints the number of games (rows), the features per team and the first and last season. For real data: 807 games, 51 features, 2004-2025.
`summary, res, keep, freq = run_all(df)`	Runs feature selection and all six models. Returns: summary = results table, res = detailed per-season results, keep = the selected features, freq = how often each feature was selected.
`os.makedirs(RESULTS, exist_ok=True)`	Creates the results folder if it does not exist.
`summary.to_csv(os.path.join(RESULTS, "summary.csv"), index=False)`	Saves the results table as results/summary.csv.
`freq.to_csv(... "feature_selection_freq.csv" ...)`	Saves how often each feature passed the p-value rule.
`pd.DataFrame({m: [x['acc'] for x in r] for m, r in res.items()}, index=SEASONS).to_csv(...)`	Builds a table of accuracy per season for each model (one column per model, one row per season) and saves it as accuracy_by_season.csv.
`make_figures(res, summary, freq)`	Draws and saves the three charts (.png) in results/.
`with open(... "selected_features.txt", "w") as f:` `  f.write("\n".join(keep) + ...)`	Writes the names of the selected features, one per line, and the line synthetic_data=False/True.
`print("\nSaved tables and figures to results/")`	Last message you see.
`if __name__ == "__main__":` `  main()`	Runs main() only when you start this file directly (py src/main.py), not when it is imported.

2. data.py (loading and preparing the data)

Code	What it does
`import os / numpy as np / pandas as pd`	os = file paths, numpy = number arrays, pandas = tables.
`ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))`	Finds the project's main folder: take this file's full path, go up two levels (src, then the repo folder). This lets the code work from any location.
`DATA_PATH = os.path.join(ROOT, "data", "bowl_games.csv")`	Full path of the dataset file.
`RESULTS = os.path.join(ROOT, "results")`	Full path of the folder where outputs are saved.
`FEATURES = [ ... ]`	Default feature names. They are used only for the fake data. With the real CSV the list is replaced below.
`SEASONS = list(range(2000, 2011))`	Default list of seasons for the fake data. Replaced below when the CSV exists.
`if os.path.exists(DATA_PATH):`	If the real CSV exists, replace both defaults using the file itself.

Code	What it does
`_hdr = pd.read_csv(DATA_PATH, nrows=0).columns`	Reads only the column names (nrows=0 means no data rows).
`FEATURES = [c[3:] for c in _hdr if c.startswith("t1_")]`	Keeps the columns that start with t1_ (team 1's features) and removes the first 3 characters, so t1_off_points becomes off_points. This gives the 51 feature names.
`SEASONS = sorted(pd.read_csv(DATA_PATH, usecols=["season"]).season.unique().tolist())`	Reads the season column, takes each distinct year once and sorts them: 2004 to 2025.
`def generate_synthetic_data(...)`	Makes fake data with the same columns, only so the code runs when the real CSV is missing. Each team gets hidden offence/defence strengths, features are noisy copies of them, and scores come from the strengths plus randomness. It is not used with your real data.
`def load_data():`	Loads the dataset.

`if os.path.exists(DATA_PATH):`
`  print("Loading real data ...")`
`  return pd.read_csv(DATA_PATH), False`
	If the CSV exists, print a message, read it into a table and return it together with False (not synthetic).

`print("... using SYNTHETIC stand-in data")` `return generate_synthetic_data(), True`	Otherwise print the warning and return the fake data with True.
`def build_xy(df, flip=None):`	Turns the table into the model's input X and target y. flip optionally swaps team 1 and team 2 in random rows.
`t1 = df[[f"t1_{f}" for f in FEATURES]].to_numpy(float)`	Takes team 1's feature columns as a number array (one row per game, 51 columns).
`t2 = df[[f"t2_{f}" for f in FEATURES]].to_numpy(float)`	The same for team 2.
`s1, s2 = df["score1"].to_numpy(float), df["score2"].to_numpy(float)`	The final scores of team 1 and team 2.
`if flip is not None:`	Only when a flip array is given (feature selection does this).
`t1, t2 = np.where(flip[:, None], t2, t1), np.where(flip[:, None], t1, t2)`	Where flip is True, swap the two teams' features: the new team 1 is the old team 2. np.where(condition, a, b) picks a where the condition is true, otherwise b.
`s1, s2 = np.where(flip, s2, s1), np.where(flip, s1, s2)`	Swaps the scores in the same rows, so features and scores stay matched.
`return np.hstack([t1, t2]), s1 - s2, t1, t2, s1, s2`	Returns X (team 1 features next to team 2 features: 102 columns), y = s1 - s2 (the margin), plus the pieces. Other code usually keeps only the first two.

3. evaluate.py (runs all models)

Code	What it does
`from feature_selection import select_differential_features`	Imports the feature selection function.
`from models import (score_regression, ..., loso, pickem_score)`	Imports the four models, the test loop and the Pick'em scoring.
`def run_all(df, verbose=True):`	Runs the whole experiment on the table df. verbose=True prints the results.
`keep, freq_table = select_differential_features(df)`	Runs feature selection. keep = the 19 chosen features; freq_table = how often each feature passed.
`if len(keep) < 2:` `  keep = FEATURES`	Safety net: if fewer than 2 features survive, use all of them.
`models = { ... }`	A dictionary of the six models, name -> model. Two baselines use all FEATURES; the other four use only the selected keep features.
`"Baseline (all feats) score reg.": score_regression(FEATURES)`	Score regression with all 51 features.
`"Baseline (all feats) diff reg.": differential_regression(FEATURES)`	Differential regression with all 51 features.
`"Score reg. (selected feats)": score_regression(keep)`	Score regression with the 19 selected features.
`"SVM regression (selected)": svm_regression(keep)`	SVM regression with the selected features.
`"Logistic (selected)": logistic_model(keep)`	Logistic regression with the selected features.

Code	What it does
`"Differential reg. (selected)": differential_regression(keep)`	Differential regression with the selected features.
`res, rows = {}, []` `rng = np.random.default_rng(7)`	res will store detailed results; rows will hold the summary rows. rng is a random number generator with a fixed seed, so random picks are repeatable.
`for name, fn in models.items():`	Goes through the models one by one.
`r = loso(df, fn)` `res[name] = r`	Tests the model with leave-one-season-out and stores the per-season results.
`ps = [pickem_score(x['pred'], x['true'])[0] / pickem_score(x['pred'], x['true'])[1] for x in r]`	For each season: the Pick'em points earned divided by the maximum possible. This is the share of the maximum for that season.
`rnd = [pickem_score(rng.normal(size=len(x['true'])), x['true'])[0] / ... for x in r]`	The same, but using random numbers as predictions: the random-guessing baseline.
`rows.append(dict(model=name, accuracy=np.mean(...), mse_margin=np.mean(...), pickem_pct_of_max=np.mean(ps), random_pickem_pct=np.mean(rnd)))`	Averages over the 22 seasons and stores one row: accuracy, margin error, Pick'em share, random Pick'em share.
`summary = pd.DataFrame(rows)`	Turns the rows into the results table.
`if verbose: print(...)`	Prints the selected features and the table (what you see in the terminal).
`return summary, res, keep, freq_table`	Gives everything back to main.py.
`def make_figures(res, summary, freq_table, outdir=RESULTS):`	Draws three charts and saves them as .png files.
`import matplotlib; matplotlib.use('Agg')`	Uses matplotlib without opening a window, so it can just save files.
`Chart 1: ax.plot(SEASONS, [x['acc'] for x in res[name]], marker='o', label=name)`	Line chart of accuracy per held-out season for four models; the dotted line at 0.5 is a coin flip. Saved as accuracy_by_season.png.
`Chart 2: ax.barh(s.model, s.accuracy)`	Horizontal bars of mean accuracy per model, with the value written beside each bar. Saved as model_comparison.png.
`Chart 3: top = freq_table ... .nlargest(20, 'm')`	Takes the 20 features chosen most often and draws their selection frequency for team 1 and team 2; the red line is the 20% threshold. Saved as feature_selection.png.

4. feature_selection.py (choosing features)

Code	What it does
`from scipy import stats`	Gives the t-distribution needed to turn a t-value into a p-value.
`def ols_pvalues(X, y):`	Fits an ordinary linear regression and returns a p-value for each feature. A small p-value means the feature really helps predict y.
`Xc = np.hstack([np.ones((len(X), 1)), X])`	Adds a column of ones at the front so the regression can have an intercept (a constant term).
`beta, *_ = np.linalg.lstsq(Xc, y, rcond=None)`	Least squares: finds the coefficients beta that make the predictions closest to y.
`resid = y - Xc @ beta`	Residuals: the real values minus the predictions (@ is matrix multiplication).
`dof = len(y) - Xc.shape[1]`	Degrees of freedom: number of games minus number of coefficients.
`sigma2 = resid @ resid / dof`	Estimated noise variance: sum of squared residuals divided by dof.
`cov = sigma2 * np.linalg.pinv(Xc.T @ Xc)`	Covariance matrix of the coefficients. pinv is a pseudo-inverse, used because many features are correlated and a normal inverse can fail.
`t = beta / np.sqrt(np.diag(cov))`	t-value of each coefficient: coefficient divided by its standard error.
`return 2 * stats.t.sf(np.abs(t), dof)[1:]`	Converts each t-value to a two-sided p-value. [1:] drops the intercept so only the features remain.
`def select_differential_features(df, n_rand=100, min_p=0.1, max_p=0.4, freq=0.2, seed=1):`	Settings: 100 random shuffles; a feature passes a shuffle if its smallest p-value is below 0.1 and its largest below 0.4; it must pass in at least 20% of shuffles.

`rng = np.random.default_rng(seed)`
`nf = len(FEATURES)`
`counts = np.zeros(2 * nf)`
	Random generator with fixed seed; nf = number of features; counts holds one counter per feature for team 1 and for team 2 (2 x 51 counters).

`for _ in range(n_rand):`	Repeat 100 times.

Code	What it does
`flip = rng.random(len(df)) < 0.5`	For every game, a random True/False: swap the two teams or not.
`X, y, *_ = build_xy(df, flip)`	Builds X and the margin y with those swaps applied.
`P = np.array([ols_pvalues(X[df.season != s], y[df.season != s]) for s in SEASONS])`	For every season s: take all games not in s, fit the regression, and collect the p-values. P has one row per season (22) and one column per feature.
`counts += (P.min(0) < min_p) & (P.max(0) < max_p)`	P.min(0) is each feature's smallest p-value across the 22 fits, P.max(0) the largest. Where both conditions hold the result is True (counts as 1) and is added to the counter.
`frac = counts / n_rand`	Fraction of the 100 shuffles in which each feature passed.
`keep = [f for i, f in enumerate(FEATURES) if frac[i] >= freq and frac[nf + i] >= freq]`	Keep a feature only if it passed in at least 20% of shuffles for team 1 (position i) AND for team 2 (position nf+i).
`return keep, pd.DataFrame({"feature": FEATURES, "team1_freq": frac[:nf], "team2_freq": frac[nf:]})`	Returns the kept features and a table of the pass fractions, which is the feature selection chart's data.

Note: the docstring still says '11' fits from the older fake data; with the real data it runs 22 fits (one per season).

5. models.py (the models and the testing)

Code	What it does
`from sklearn.linear_model import LinearRegression, LogisticRegression`	The linear regression and logistic regression models.
`from sklearn.svm import SVR`	Support vector regression.
`from sklearn.preprocessing import StandardScaler`	Rescales each feature to mean 0 and spread 1.
`from sklearn.pipeline import make_pipeline`	Chains steps (scale, then model) into one object.
`def cols(feats):` `  return [FEATURES.index(f) for f in feats]`	Returns the column position of each chosen feature in the full feature list.
`def loso(df, fit_predict_fn):`	Leave-one-season-out test. fit_predict_fn is a function that trains on a training table and predicts a test table.
`for s in SEASONS:`	Go through the 22 seasons; each one is the test season once.
`tr, te = df[df.season != s], df[df.season == s]`	tr = all games not in season s (training). te = games of season s (testing).
`pred = fit_predict_fn(tr, te)`	Trains on tr and returns predicted margins for te.
`true = (te.score1 - te.score2).to_numpy()`	The real margins of the test games.
`acc=np.mean((pred > 0) == (true > 0))`	pred > 0 means the model says team 1 wins; true > 0 means team 1 really won. == checks whether they agree for each game; the mean of True/False is the share correct. This is the accuracy.
`mse=np.mean((pred - true) ** 2)`	Mean squared error: average of the squared difference between predicted and real margin.
`out.append(dict(season=s, acc=..., mse=..., pred=pred, true=true))` `return out`	Stores this season's results and returns all 22.
`def score_regression(feats):`	MODEL 1. Predicts each team's points, then subtracts.
`off_f = [f for f in feats if f.startswith("off_")]`	The offensive features among the chosen ones (a team's own stats).
`def_f = [f for f in feats if f.startswith("def_")]`	The defensive features (what opponents did against the team).
`def design(d, me, opp):`	Builds the input table for predicting the points of team 'me' against team 'opp'.
`np.hstack([d[[f"{me}_{f}" for f in off_f]].to_numpy(float), d[[f"{opp}_{f}" for f in def_f]].to_numpy(float)])`	Joins side by side: my offensive features and the opponent's defensive features. For me = t1 this is t1_off_... and t2_def_....
`def fp(tr, te):`	The function loso will call: train on tr, predict te.
`X = np.vstack([design(tr, "t1", "t2"), design(tr, "t2", "t1")])`	Stacks two blocks: every game once from team 1's side and once from team 2's side. So each game gives two examples of 'a team's stats -> its points'.
`y = np.concatenate([tr.score1, tr.score2])`	The matching targets: team 1's points then team 2's points, in the same order.
`m = LinearRegression().fit(X, y)`	Trains a linear regression to predict points.
`return m.predict(design(te, "t1", "t2")) - m.predict(design(te, "t2", "t1"))`	Predicts team 1's points and team 2's points and subtracts: the predicted margin.
`return fp`	Returns the function so it can be used later.
`def differential_regression(feats, flip_seed=None):`	MODEL 2. Predicts the margin directly. (flip_seed is unused.)
`idx = cols(feats) + [len(FEATURES) + i for i in cols(feats)]`	Column positions to use in X: the chosen features' positions for team 1, plus the same positions shifted by 51 for team 2 (team 2's columns come after team 1's).
`X, y, *_ = build_xy(tr)`	X (102 columns) and y (the margin) for the training games.
`m = LinearRegression().fit(X[:, idx], y)`	Trains a linear regression on only the chosen columns.
`Xt, *_ = build_xy(te)` `return m.predict(Xt[:, idx])`	Builds the test inputs the same way and returns the predicted margins.
`def logistic_model(feats):`	MODEL 3. Predicts who wins, not by how much.

Code	What it does
`make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(X[:, idx], y > 0)`	Rescales the features, then trains logistic regression. y > 0 is True when team 1 won. max_iter=1000 allows up to 1000 steps while it learns.
`return m.decision_function(Xt[:, idx])`	Returns a score for each game: above 0 means team 1 is predicted to win, and the larger |score|, the more confident. It is not in points.
`def svm_regression(feats):`	MODEL 4. Support vector regression on the margin.
`make_pipeline(StandardScaler(), SVR(C=1.0)).fit(X[:, idx], y)`	Rescales features, then trains SVR. C=1.0 sets how strongly errors are punished. The default RBF kernel allows a curved (nonlinear) fit.
`def pickem_score(pred, true, rng=None):`	Simulates the Pick'em game. (rng is unused.)
`n = len(pred)`	Number of games.
`pts = np.empty(n)`	Empty array that will hold each game's points.
`pts[np.argsort(np.abs(pred))] = np.arange(1, n + 1)`	np.abs(pred) is the confidence. argsort lists the games from least to most confident. The least confident game gets 1 point, the next 2, ... the most confident n.
`correct = (pred > 0) == (true > 0)`	True for games whose winner was predicted correctly.
`return float((pts * correct).sum()), n * (n + 1) / 2`	Total points of the correct picks (wrong ones count 0), and the maximum possible n(n+1)/2.

Note: in the results table the mse_margin value for the Logistic row is not a real error in points, because the logistic model outputs a score and not a margin. Compare mse_margin only across the regression models.

6. download_data.py (run once to build the CSV)

fetch() calls the CollegeFootballData API with your key, caches each answer in data/raw/ so it is not downloaded twice, retries on network errors, and stops with a clear message for a wrong key (401/403) or call limit (429). **parse_stat()** turns values like '15-25' into two numbers and 'mm:ss' into seconds. **season_team_features()** downloads every regular-season game week by week and averages, per team, its own stats (off_) and its opponents' stats (def_). **bowl_games()** downloads the postseason games with final scores. **build()** keeps only stats available for at least 90% of teams, joins each bowl game with both teams' season averages, and builds the table. **main()** reads the key from the CFBD_API_KEY environment variable and writes data/bowl_games.csv.

7. Questions a teacher may ask

What does fit/predict mean? fit trains the model on known examples; predict applies it to games it has not seen.
Why leave-one-season-out? The test season is never in training, which matches predicting a new season.
Why shuffle team 1 and team 2? Their order is arbitrary; shuffling stops a feature looking useful just because of its position.
Why StandardScaler for SVM and logistic only? They are sensitive to the scale of features; plain linear regression is not.
Why abs(pred) for confidence? A larger predicted gap means a surer pick.
Any weakness? Feature selection used all seasons, so the selected-feature scores are slightly optimistic.